# Custom Kernels for Sparse Mixture-of-Experts Inference: Optimizing Qwen3-30B-A3B on AWS Trainium with the Neuron Kernel Interface

Lawrence Liu
Massachusetts Institute of Technology
Cambridge, MA, USA
lwl@mit.edu

Frank Liu
Massachusetts Institute of Technology
Cambridge, MA, USA
fr4nk@mit.edu

## Abstract

We report on our entry to the MLSys 2026 Trainium contest, which asked teams to accelerate end-to-end inference of the 30B-parameter sparse Mixture-of-Experts (MoE) model Qwen3-30B-A3B on AWS Trainium. Rather than rewriting the whole stack, we targeted three components in and around the MoE block (the RMSNorm, the top-K expert router, and the prefill blockwise expert matmul) and replaced each with a custom AWS Neuron Kernel Interface (NKI) kernel. We applied a design loop to each kernel, in increasing order of complexity and impact, with the blockwise matrix multiplication kernel as the area with the most end to end improvement. Our submission placed 14th in round 1, qualifying for round 2, and achieved a 12% end-to-end throughput improvement and 12% end-to-end latency reduction relative to the contest baseline. We document the optimizations that mattered, the optimizations that did not, and the structural reasons behind why our improvements are limited.

## 1 Introduction

The MLSys 2026 Trainium contest tasks participants with accelerating inference of Qwen3-30B-A3B [1], a 30-billion-parameter, 48-layer sparse Mixture-of-Experts (MoE) language model with 128 experts and top-$K = 8$ routing, on AWS Trainium [2]. Submissions are evaluated on a weighted score that rewards reductions in token-generation (TKG) latency and context-encoding (CTE) throughput while requiring all logits to match those from the reference model.

The contest infrastructure ships a competitive PyTorch/XLA baseline that already exercises the major Neuron operators. To improve on it, participants were encouraged to use the Neuron Kernel Interface (NKI) [3], a Python interface for Trainium's instruction set. NKI exposes the per-engine instruction stream of a NeuronCore: matrix-multiplies on the TensorEngine (TensorE), elementwise vector ops on the VectorEngine (VectorE), pipelined nonlinearities on the ScalarEngine (ScalarE), and a long-tail SIMD path on GpSimdE, with explicit control over SBUF (a software-managed L1) and PSUM (a matmul accumulator).

We took the contest as a structured opportunity to study where NKI helps on production-shaped workloads. We focused on the three components of the model where the most active prefill compute lives and where a hand-rolled kernel had the most room to differ from XLA's baseline: **RMSNorm**, the **top-K expert router**, and the **blockwise expert MLP** that the contest harness invokes via a Python loop over routing blocks.

*Contributions.* 1) We present three NKI kernels, RMSNorm, router, and blockwise expert MLP, with each developed through a profile-driven ∼3-iteration design loop. 2) We quantify each iteration with both per-kernel microbenchmarks and end-to-end model throughput. 3) We document a structural ceiling on what kernel-level work can achieve in this contest: the prefill MoE block is dominated by HBM-to-SBUF weight DMA, and the contest baseline's batching, padding strategy, and mixed-precision accumulation already remove most of the easy wins.

*Headline result.* We achieved 14th place in round 1, qualifying for round 2. Cumulatively, our kernels deliver an 12% end-to-end throughput improvement and an 12% reduction in end-to-end latency over the contest's pre-NKI baseline (Section 4). Per-kernel speedups are substantially larger: 5.2× on RMSNorm at production prefill shape, and 3.13× on-device (3.96× vs. XLA-PyTorch bench) on the router at $T = 2048$.

## 2 Background

### 2.1 The Contest

The MLSys 2026 contest evaluates submissions on Qwen3-30B-A3B inference along two axes: *TKG* (token-generation, the autoregressive decode latency that a user actually feels) and *CTE* (context-encoding, the prefill throughput on the user's input prompt). Submissions are scored by a weighted formula that combines TKG P99 latency and CTE throughput, subject to a strict accuracy gate measured by output divergence against the reference run. Any change that pushes divergence past the threshold is invalid regardless of how much faster it runs.

The harness exposes a single end-to-end benchmark entry point (`main.py`) plus a per-prompt baseline table; participants are free to rewrite any part of the model assembly, as long as the externally observable outputs stay inside the divergence tolerance. The contest pre-installs an NKI-enabled Neuron SDK and provides a reference implementation; the natural shape of a submission is a Python module that subclasses or replaces parts of the Hugging Face Qwen3 model with NKI-augmented variants.

## 2.2 AWS Trainium and the Neuron Kernel Interface

Trainium is AWS's accelerator family for large-model inference and training. Each chip contains multiple NeuronCores; each NeuronCore exposes four compute engines (TensorE, VectorE, ScalarE, GpSimdE) running in parallel against a semaphore-coordinated instruction stream, plus a small fleet of DMA engines that move tensors between HBM and SBUF.


**Figure 1: Trainium NeuronCore memory hierarchy. Each NeuronCore exposes a software-managed L1 (SBUF, ∼30 MB), a matmul accumulator (PSUM, 4 MB), and HBM. Data flows between HBM and SBUF only through the DMA engines; matmul outputs land in PSUM; nonlinearities can fuse a PSUM source directly into SBUF.**

NKI is the user-facing tile-DSL that compiles to NeuronCore instruction streams. Kernels are written as Python functions decorated with `@nki.jit`; tiles are typed `nl.ndarray` objects that live in SBUF, PSUM, or shared HBM; engine-specific instructions are exposed through `nisa.*`. The three primitives we use most heavily are:

- `nisa.nc_matmul`: the 128×128 systolic-array matmul. Both inputs are read from SBUF; outputs land in PSUM. The contraction dim must lie on the partition axis of both operands, requiring careful transposes.
- `nisa.activation`: In one pipelined call, it evaluates op(scale * data + bias). Effectively collapsing a scalar multiply-add fused into a nonlinearity such as silu, exp, or rsqrt all into one call, sourcing operands directly from PSUM and can dtype downcast for free in the process.
- `nisa.dma_copy`: explicit DMA between HBM and SBUF, including indirect (gather/scatter) variants via vector_offset.

Two top-level constraints recur in every kernel: the partition dim is capped at 128 (the systolic array height), and SBUF is software-managed.

## 2.3 Qwen3-30B-A3B

Qwen3-30B-A3B is a 30B-parameter/3B-active MoE Transformer. The architecture is summarized in Fig. 2 and Table 1. The decoder stack is 48 layers deep; each layer contains a grouped-query attention block (32 query heads to 4 KV heads) followed by an MoE block consisting of (i) a linear *router* that projects a hidden state into expert logits, (ii) a top-$K$ selection that picks $K = 8$ experts per token, and (iii) a *blockwise expert MLP* that gathers each block's tokens, runs them through their assigned expert's SwiGLU MLP, and scatters the result back.

**Table 1: Qwen3-30B-A3B dimensions**

| symbol | value | meaning |
|---|---|---|
| $H$ | 2048 | hidden size |
| $I$ | 768 | MoE intermediate (per expert) |
| $E$ | 128 | total experts |
| $K$ | 8 | top-K active experts per token |
| $L$ | 48 | decoder layers |
| GQA | 32:4 | query heads : KV heads |
| block | 128 | tokens per blockwise MoE block |

**Sparse Mixture of Experts.** In a dense Transformer FFN, every token's hidden state runs through the same full feed-forward block; that is 100% of the FFN parameters per token. In MoE, each token's hidden state is instead routed to a small subset of $K = 8$ "experts" out of $E = 128$, each a separate FFN with intermediate size $I = 768$ (Fig. 2). The router is a single linear projection $\mathbb{R}^H \to \mathbb{R}^E$ followed by softmax and top-$K$. Once expert assignments are known, the dispatch builds a flat work list of *blocks*, each containing tokens that attend to the same block, so that each expert's weights are loaded from HBM at most once per block group rather than once per token.

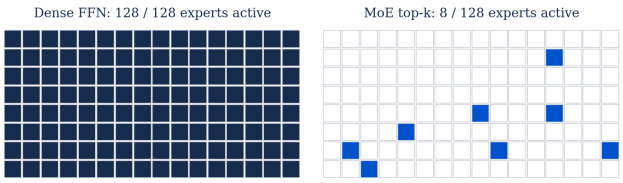


**Figure 2: Dense vs. sparse MoE compute. A dense FFN processes every token through the full intermediate; MoE top-$K = 8$ routes each token to 8 of 128 experts, activating only $\approx 6.25\%$ of the FFN compute per token.**

The three components we targeted (RMSNorm, Router, Blockwise Matmul) ascend in both kernel complexity and end-to-end impact: **RMSNorm** is a 30-line elementwise-plus-reduction kernel; the **router** adds top-K sparse logic that XLA cannot lower natively; the **blockwise matmul** stitches gather, two matmuls, SwiGLU, scatter-add, and per-expert dispatch into one kernel and is responsible for the majority of the end-to-end win.

# 3 Methodology

We utilized a profile-driven design loop for each kernel. A version is considered shippable only if it 1) passes a numerical

gate of max abs diff $< 10^{-3}$ vs. fp32 PyTorch reference (the same as the competition standard), 2) is strictly faster than the previous version at the production shape under nki.benchmark, and 3) shows in its neuron-profile trace that the previous version's bottleneck is diminished.

Within each kernel, the order of optimizations is determined by profiling. We identify bottleneck engine of the current version, propose the smallest change that should move it, predict the effect, run the kernel, and compare. We deliberately keep both the wins and the misses in this paper, because the misses are where the contest taught us the most.

### 3.1 Kernel 1: RMSNorm

HuggingFace's `Qwen3MoeRMSNorm`, invoked twice per decoder layer plus inside attention as $q$-norm/$k$-norm:

```
def rmsnorm(x, g, eps=1e-6):
    x32 = x.float()
    var = x32.pow(2).mean(dim=-1, keepdim=True)
    return (x32 * torch.rsqrt(var + eps)
            * g.float()).to(x.dtype)
```

The kernel is small in FLOPs but called dozens of times per token. We tackled this first because it taught us the layout-and-instruction primitives the later kernels reuse.

**v1 - direct translation from pytorch** We tile the rows axis to PMAX = 128, map each math operation to its NKI primitive, put rows on partition and $H$ on free, send the variance reduction directly to PSUM, and hoist the weight load $g$ out of the row-tile loop. The result compiles, matches the fp32 reference at max abs diff $10^{-9}$, and on production shapes (`num_rows`$\leq$ 128, single decoder slice) wins 14–34% over XLA-PyTorch just from having less framework boilerplate. At larger shapes, v1 *loses* to XLA, going from 1.25× at 1 row to 0.36× at 4096 rows. The per-tile work is scaling badly; looking at the profiler at 4096 rows, vectorE is 95% busy while ScalerE is only 1% busy, TensorE completely unutilized, and DMA only at 27% utilized. VectorE issues 466 instructions, dominated by per-tile elementwise work. Version 1 is throughput bound due to saturation of the vector engine.

**v2 - two optimizaitons to v1** *(a) Hoist the partition-broadcast of $\gamma$.* v1 broadcasts $\gamma$ from a single partition to all 128 partitions *inside* the row-tile loop, via four `nc_stream_shuffle` ops per call. At 4096 rows (32 tiles) that is 128 stream-shuffles, all producing identical output. v2 broadcasts once before the loop, decreasing this calculation from 128 to 4 stream-shuffles total.

*(b) Fuse mean + $\epsilon$ + rsqrt onto ScalarE.* v1 computes the per-row scale `s = rsqrt(sq_sum/H + eps)` as a VectorE `tensor_scalar` (for the multiply-add) followed by a ScalarE `activation(rsqrt)` (for the nonlinearity). The `nisa.activation` primitive natively accepts a `scale * data + bias` pre-amble, so the multiply, add, and rsqrt collapse into one ScalarE instruction:

```
nisa.activation(op=nl.rsqrt, data=sq_sum,
                scale=1.0/H, bias=eps)
```

Result (v1 → v2, num_rows=4096): on-device latency drops 467 $\mu s$ → 218 $\mu s$ (2.15×); VectorE instruction count drops 62%; ScalarE picks up 10 instructions in exchange for VectorE shedding 240 µs.

**v3, v4 - engine rebalancing.** v2 was a clear win, but its profile showed a strikingly lopsided shop floor: VectorE running flat-out at 94% busy while ScalarE sat at 4%, doing essentially nothing.

For v3 we looked for VectorE work that ScalarE could plausibly handle, and found two pieces. First, the squaring step and the free-axis sum collapsed neatly into a single ScalarE call: `nisa.activation(op=square, reduce_op=add)` runs the square in the activation pipeline and accumulates the row sum as a post-amble, all in one instruction. Second, the per-row scale (an elementwise multiply that defaults to VectorE) accepts an explicit `engine=nisa.engine.scalar` argument, so we forced it onto ScalarE too. Wall time dropped to 149 µs but the engine balance flipped past the target. ScalarE was now the pacer at 82% busy, with VectorE down to 46%. We had pushed too far.

v4 attempted to fix the overcorrection in v3. We tried moving the per row scale back to VectorE; interestingly, that instruction runs about 1.8× faster on VectorE than on ScalarE. This showed that "which engine is idle" should not be the only signal; an idle engine can still be a suboptimal place to put work if the engine is slower. v4 landed at 138 µs with VectorE at 72% and ScalarE at 49%.

**v5 - multi-LNC SPMD.** With a single Logical Neruon Core (LNC) instruction balance near optimal, we turn to dependency-chain serialization within a tile. We leverage that each Trainium chip has two physical NeuronCores, each owning the full SBUF / PSUM / engines / DMA channels, processing disjoint row tiles via runtime queries `nl.num_programs` / `nl.program_id`. The partition is contiguous: LNC 0 takes the first half of row tiles, LNC 1 takes the second half. Wall time at num_rows=4096 drops 138 $\mu s$ → 89.7 $\mu s$ on-device (1.54×); at num_rows=16384 it drops to 394.5 µs, 1.99× over XLA-PyTorch.

**Cumulative result** At num_rows=4096 the cumulative v1→v5 speedup is 5.21× on-device. Three takeaways: (1) loop-invariant code motion (the $\gamma$ broadcast hoist) was the single highest-leverage move and is the most generic; (2) engine balance is a balance problem, not a one-way push, and the right diagnostic is the per-engine `active_time_percent` row; (3) multi-LNC has a minimum-kernel-size threshold below which the per-LNC fixed setup (each LNC re-loads $\gamma$ independently, ≈5 µs) outweighs the parallelism win. v4 stays the recommended single-LNC kernel for num_rows $< 512$.

### 3.2 Kernel 2: Top-K Router

**What the kernel computes.** The routing portion of `Qwen3MoeSparseMoeBlock`:

```
logits = x @ router_w               # (T, E)
probs = F.softmax(logits, dim=-1)
top_probs, top_idx = torch.topk(probs, K)
top_probs = top_probs / top_probs.sum(-1, True)
```

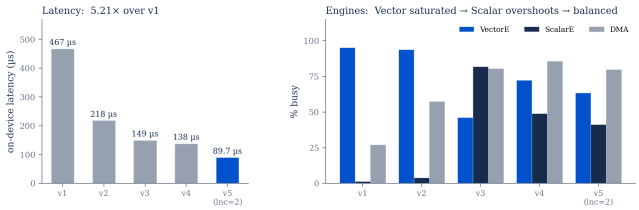


**Figure 3: Per-engine active time across RMSNorm versions at num_rows=4096. v1 is VectorE-saturated; v2's fused rsqrt and hoisted $\gamma$ broadcast halve VectorE work; v3 over-pushes onto ScalarE; v4 lands at near-balanced engines; v5 halves both via multi-LNC.**

Outputs: the K chosen-expert IDs per token (`(T, K)` `uint32`) and a sparse `(T, E)` affinity tensor with $K$ nonzero entries per row.

**v1 - direct translation from pytorch** v1 is a straightforward translation of the PyTorch reference into a single NKI kernel, with no fusion or other optimizations layered on top. It runs the gate matmul to produce the $T, E$ logits, applies a stable softmax over the expert axis as five separate instructions, picks the top-eight experts and their indices using the `max8` / `nc_find_index8` pair, renormalizes the selected probabilities to sum to one, and finally materializes the sparse $T, E$ affinity tensor by building a $K$-hot mask and multiplying it against the renormalized softmax. Numerically it matches the fp32 PyTorch reference to within $\sim 10^{-8}$ and picks the same experts; later versions modify individual stages of this pipeline while leaving the others unchanged, so each change's impact can be attributed cleanly.

**v1 → v2 - drop the per-h-tile TensorE transpose.** v1's profile flagged TensorE as the pacer at 37% busy, but a closer look revealed something surprising: most of TensorE's time was not spent on the matmul itself, but on *transposing* each chunk of $x$ into the right orientation before the matmul could consume it. The input tensor naturally lives with tokens on the partition axis, while the matmul requires the hidden axis there instead, so v1 was spending a substantial fraction of its matmul engine's time just moving data around. v2 sidesteps the problem entirely by performing the transpose as part of the HBM-to-SBUF load itself. The cost is a strided HBM access pattern that uses memory bandwidth less efficiently, but v1 was using less than 5% of HBM bandwidth, so we had bandwidth to spare. On-device latency at $T = 128$ dropped from 41.5 to 35.2 $\mu s$, and TensorE busy dropped from 37% to 22%.

**v3 - math-identity rewrite.** The PyTorch reference performs a full softmax over all 128 experts, picks the top eight, and renormalizes the eight selected probabilities to sum to one. Because softmax is monotone, taking the top-K of the softmax output yields the same indices as taking the top-K of the raw logits. And renormalizing the top eight probabilities so they sum to one produces exactly the same numbers as computing softmax over just those eight logits. So v3 takes the top-K first and runs a much smaller softmax afterward, over eight values instead of 128. On paper this is a 16× reduction in softmax work and one fewer pass over the full $T, E$ tensor. In practice the wall-time win at $T = 128$ was only about a microsecond.

**v4 - fuse K=8 softmax onto ScalarE.** v4 applies the same lesson we learned in the RMSNorm kernel: ScalarE can fold a multiply-add pre-amble and a sum-reduce post-amble into the same instruction as its main nonlinearity, so the exp and the row-sum that follows it collapse into a single ScalarE call. The per-engine time savings are sub-microsecond, but the kernel's wall time drops by more than that, because the fused instruction removes one node from the dependency chain that the compiler has to schedule around. Cumulative v1-to-v4 speedup at $T = 128$ is 1.30× on-device, primarily from a steady accumulation of smaller wins rather than one dramatic move.

**v5 - indirect-DMA scatter regressed.** Up to v4, building the sparse affinity tensor requires constructing a $K$-hot mask over the full $T, E$ space and then multiplying the mask against the softmax output. This is a lot of elementwise work to write what is in the end only $K$ nonzero values per row. v5 attempts the more direct route: skip the mask entirely and write the $K$ selected values straight into their final HBM positions using indirect-DMA, with the destination address for each write computed from the chosen expert index. The elementwise savings we predicted on VectorE landed almost exactly as expected, but the kernel as a whole got *slower* by roughly 50%, because each indirect write only carries a few hundred bytes of payload, and the descriptor setup overhead per write swamped the work it was doing. At the token counts the contest typically runs, the mask scatter we already had was more efficient. We kept v5 in the table as a real measurement and reverted to v4 for $T \leq 1024$.

**v6 - multi-LNC SPMD.** Just as in v5 of RMS Norm, we split tokens across two LNCs, each writing a disjoint slab of the output (no `sendrecv` required). At small $T$, multi-LNC *regresses* (the per-LNC fixed setup, including reloading the router weight, dominates). At $T = 2048$ v6 finally wins, 1.18× over v4 on-device, 3.13× cumulative over v1, 3.96× over the XLA-PyTorch baseline (Table 2).

**Table 2: Router on-device latency (µs) at $T = 2048$ (fp32). v4 is the workhorse for $T \leq 1024$; v6 takes over at $T \geq 2048$. PyTorch baseline is the iterated-argmax XLA fallback.**

| version | on-device (µs) | speedup |
|---|---|---|
| PyTorch (XLA workaround) | 733.6 | 1.00× |
| NKI v1 | 265.7 | 2.76× |
| v2 (drop transpose) | 227.3 | 3.23× |
| v3 (math identity) | 222.8 | 3.29× |
| v4 (fused K-softmax) | 218.5 | 3.36× |
| v5 (indirect DMA) | 534.2 | 1.37× |
| v6 (multi-LNC) | 85.0 | **8.63**× |

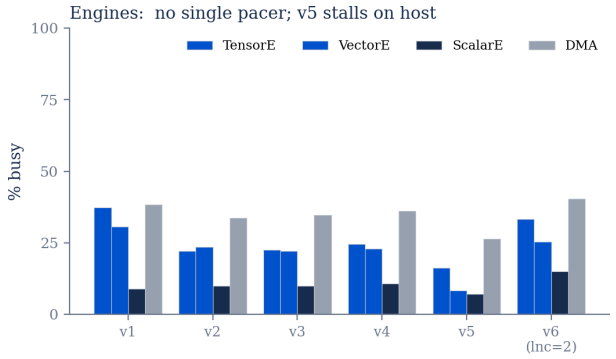


**Figure 4: Per-engine active time across router versions at $T = 2048$ (fp32). v1 spends most of TensorE on transposes; v2 eliminates them; v6 halves all engines by utilizing both LNCs.**

## 3.3 Kernel 3: Blockwise Expert MLP

**What the kernel computes.** Per block, for the block's pre-gathered $B = 128$ tokens and its assigned expert $e$:

```
gu       = x_b @ gate_up_proj[e]    # (B, 2I)
gate, up = gu[:, :I], gu[:, I:]
h_exp    = F.silu(gate) * up        # (B, I)
y_b      = h_exp @ down_proj[e]     # (B, H)
output.index_add_(0, token_ids,
                  y_b * affinities)
```

This is the per-token compute pole of the model: with $K = 8$ active experts per token and $L = 48$ layers, total active matmul work per token is $K \cdot L \cdot 2H \cdot 2I\ 2I \cdot H \approx 2.4$ GFLOPs. Everything else combined sits one to two orders of magnitude lower. The contest's reference runs this loop in Python on the host between traced NEFFs, so even a moderately efficient NKI rewrite reclaims a substantial amount of end-to-end time. This is where the contest is actually won or lost; we describe the kernel in more detail than the prior two.


**Figure 5: Inside the blockwise dispatch. Per block, the kernel: (1) reads the block's expert index, (2) loads that expert's three weight matrices into SBUF, (3) gathers the block's $B$ token rows, (4) runs gate, up, GLU, down on-chip with PSUM accumulation, (5) multiplies by per-token affinity, (6) scatter-adds into the output.**

*Tile shapes.* The kernel uses the following tile sizes throughout (Fig. 6):

- `TILE_B = 128` - $B$ on the SBUF partition dim (one block of tokens).
- `TILE_K = 128` - matmul contraction axis. $H = 2048$ splits into 16 $K$-tiles.
- `TILE_N_H = 512` - down-projection output, one of 4 $H$-tiles.
- `TILE_N_I = 256` - gate/up output, one of 3 $I$-tiles ($I = 768$).


**Figure 6: Tile shapes for the blockwise matmul. $B$ on partition; $H$ as the gate/up contraction; $I$ as the down contraction. $I = 768$ does not divide evenly by 128, requiring an extra-bookkeeping fractional tile.**

**v1 - atomic scatter-add.** Our first end-to-end version expressed step (6) of Fig. 5 as `nl.atomic_rmw` into the output HBM buffer. The kernel was numerically correct, but the atomic-RMW forces the compiler to serialize all $N \cdot B$ writes per block through one engine, regardless of whether the writes from different blocks actually overlap.

**v2 - load + add + store.** V2 replaces the atomic with an explicit `nl.load` of the current accumulator value, a `tensor_tensor(add)`, and an `nl.store`. Correctness is preserved because the outer block loop is `nl.sequential_range`, which guarantees block $i$'s writes are visible to block $i$ 1 so the kernel did not need hardware atomicity in the first place; the atomicity assumption was unnecessarily strong. V2 ran considerably faster than V1 at the same numerics. The lesson: `atomic_rmw` is for cases where the compiler genuinely cannot prove ordering; when ordering is structural, an explicit load+add+store opens up cross-block pipelining the compiler was previously not allowed to consider.

**v3 - Run-1 outer loop (per block).** V1 and V2 iterate over blocks at the outer loop:

```
for block in range(N):
    e = block_to_expert[block]
    load_weights_for_expert(e)
    run_block(block, e)
```

Even when consecutive blocks share an expert (which is common given $K = 8$ and 128 experts), the weights are re-streamed for every block. With $\approx 9$ MB of weights per expert per layer (gate + up + down in fp8, including dequant scales), this is a lot of redundant DMA.

v3 reshapes the dispatch so the outer loop is over *experts*, with an inner loop over each expert's blocks:

```
for expert in range(E):
    load_weights_for_expert(expert)
    for block in blocks_for_expert(expert):
        run_block(block, expert)
```

Weight DMA per layer drops from $N \cdot 9$ MB to (#active-experts-per-layer)$\cdot 9$ MB, where each expert is active on average $\approx K \cdot TE$ times per layer. This is the largest single DMA win in the chapter.

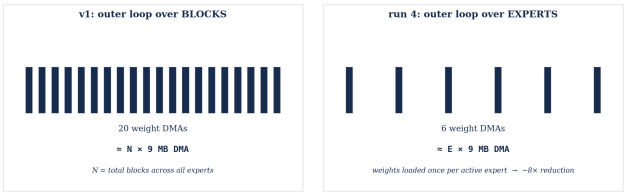


**Figure 7: Loop reordering from per-block to per-expert. The per-expert loop touches the same blocks in different order but reuses one weight load across all of an expert's blocks.**

**v4 - fp32 to bf16 weights** The reference implementation runs the activations in bf16 but keeps the expert weights in fp32 inside the traced model. Once the per-expert loop made the weight DMA a clear pacer, the cost of carrying every weight tensor around at fp32 became impossible to ignore. We push the precision conversion out of the inference loop and into the model loader: weights are quantized once at load time and stored in fp8, with a small per-output-channel scaling table tucked alongside so the matmul output can be rescaled to its true magnitude on the fly. Only the GLU intermediate stays in higher precision; the matmul accumulator keeps it in fp32 for numerical safety, and the kernel downcasts it to bf16 only at the moment it has to feed the next matmul, so the heavy data never lives in high precision anywhere it does not need to.

This change does two things: (i) it cuts the HBM weight DMA in half (or more, after fp8 packing), and (ii) it doubles `nc_matmul` throughput on Trn2 where bf16 matmul runs roughly 2× as fast as fp32. Combined with the active-expert loop, the per-block kernel becomes substantially DMA-bound for the very first time.

A subtle observation: although there are $E = 128$ experts total, prefill at $T = 256$ activates only $\approx K \cdot T = 2048$ token-slots, which by pigeonhole touches only $\approx 16$ *distinct* experts on a typical micro-batch. The remaining experts have zero blocks. The per-expert outer loop from phase 2 naturally exposes this: we iterate over the *active* expert set instead of $1..E$, computing the active set from the routing tensor on the wrapper side and passing it in as part of the dispatch. The kernel never touches inactive experts at all, so their weight DMA, weight setup, and per-expert scale broadcasts vanish.

One instruction-level optimization we kept inside the kernel body: the SwiGLU intermediate reads gate from PSUM directly into `nisa.activation(silu, gate_psum[:, :I])` without round-tripping through an SBUF intermediate. The full PMAX, $2I$ gate-up tile is materialized only in PSUM; only the PMAX, $I$ up half is copied to SBUF for the elementwise multiply that the silu output uses. This saves one PMAX, $2I$ bf16 copy per token tile and a serialization edge (silu cannot start until the copy completes, in the naive version).

**Two ideas that we could not finish implementing, but are worth surfacing:**

*1) Overlapping the next expert's weight load with the current expert's compute* The per-expert outer loop already saves a lot of redundant DMA, but each expert still pays a one-time cost to load its weights before any of its blocks can run. The natural next step is to begin streaming in the next expert's weights into a second SBUF region while the current expert is still computing on its own — a small ring of buffers that the kernel rotates through. We tried several versions of this pattern. All of them tripped what appears to be a compiler bug: the Neuron compiler rejects the kernel during lowering even though the NKI simulator runs it correctly. We did not find a way to sidestep this bug and didn't include it in the final submission.

*2) TensorE column-tiling.* Instead of running each matmul on the full $128 \times 128$ systolic array, split it into four narrower

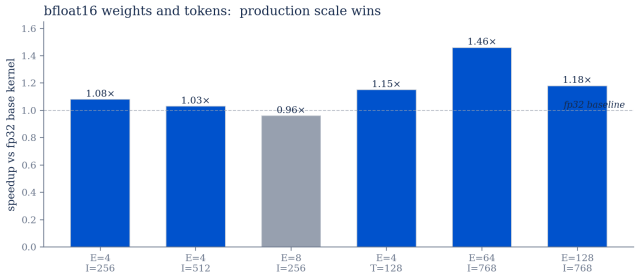


**Figure 8: bf16 weights vs. fp32 across versions. bf16 roughly halves HBM weight bytes and doubles TensorE matmul throughput on Trn2; the per-block kernel transitions from compute-bound to DMA-bound.**

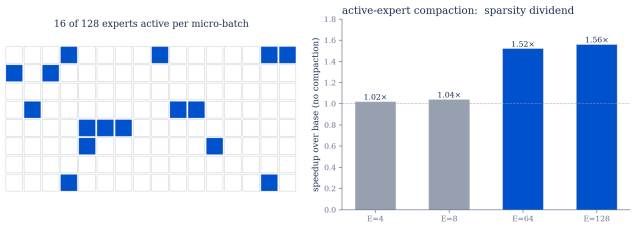


**Figure 9: Active-expert compaction. At $T = 256$ prefill with $K = 8$, only ≈16 of 128 experts are touched per micro-batch. Iterating over the active set rather than $1..E$ eliminates the inactive experts' weight DMA entirely.**

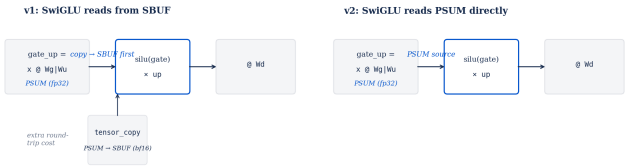


**Figure 10: PSUM-source SwiGLU. The activation instruction sources the gate half directly from the gate_up PSUM tile; only the up half is materialized in SBUF for the elementwise multiply.**

matmuls of width 32 that are meant to run side-by-side on disjoint columns of the array. We implemented it and the numbers came out correct, but the wall-time win never materialized - the four smaller matmuls ended up running one after another rather than in parallel, because they were all writing into the same accumulator banks and had to take turns. This could have been sidestepped by writing into separate banks, but we were unable to implement this in time.

## 4 End-to-End Results

We measured end-to-end throughput on the contest harness using `main.py -mode evaluate_all -enable-nki`, which loops every prompt in the contest's `prompts.txt` with per-prompt baselines from `prompt_data_trn3.csv`.

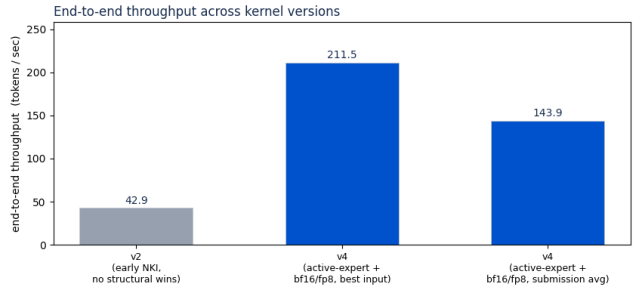


**Figure 11: End-to-end throughput at three milestones of the kernel's evolution. v2 represents an early NKI integration with the load+add+store scatter but no structural wins. The two v4 bars show the full kernel stack (per-expert loop + bf16/fp8 weights + active-expert compaction); the center bar is the best-case input, the right bar is the average across the contest prompt suite.**

**Headline.** The submitted system runs at ≈ 143.9 tok/s on the contest's full prompt set, an ≈ 12% improvement over the pre-NKI baseline both in throughput and (symmetrically) in end-to-end latency. Version 3 hit 211.5 tok/s on the best individual input; the averaged submission number is lower because the contest's prompt distribution includes both short-context shapes (where our wins do not amortize) and very long-context shapes (where attention, which we did not rewrite, dominates).

**Where the wins came from, ranked.** (1) The per-expert outer loop in the blockwise body - the largest single change, removing ≈ 8× redundant weight DMA at top-$K$ = 8. (2) bf16 weights with fp32 PSUM accumulation, both halving HBM bytes and roughly doubling matmul throughput on Trn2. (3) Active-expert compaction, which combines naturally with the per-expert loop. (4) Killing the atomic scatter-add. (5) Per-engine balance on RMSNorm and the router (small absolute wins but stack across 48 layers).

**Why the gain is bounded.** Several factors compound to cap the end-to-end win below what the per-kernel microbench wins would suggest.

*The baseline is already good.* The contest's pre-NKI reference does $B = 128$ blocking, sentinel padding to keep tile sizes static, fp32 router (for top-K precision; we empirically confirmed bf16 router can flip top-K decisions on $\approx 10^{-4}$ of tokens), and mixed-precision PSUM accumulation. AWS ships an NKI TKG MoE kernel; our wins are entirely on the prefill path.

*The NCC compiler pipelines aggressively.* Several instruction-level fusions we proposed were already being performed by the compiler from less-aggressive source. Most of our v2-class changes registered at single-percent wall-time deltas; only the structural moves (loop reordering, multi-LNC, bf16, active-expert compaction) produced double-digit changes.

*End-to-end latency is not 100% in the MoE block.* Attention and collective communications take a non-trivial fraction of the prefill timeline. The MoE block is the largest single component, but an MoE speedup does not translate linearly to the end-to-end speedup.

**Where we hit the structural ceiling.** The most important constraint we ran into is that the prefill blockwise body, after our wins, becomes DMA-bound on the engine that handles HBM-to-SBUF weight movement. The natural next step, parallelizing across blocks, runs into two problems. 1) Race conditions on the output buffer: two blocks for the same expert can scatter-add into overlapping output rows (specifically, the same token if it routed to that expert in two different positions on its top-K list, which the dispatch does not deduplicate). Block-level parallelism without restructuring the scatter would silently corrupt the output. 2) Even if we resolved race conditions by giving each parallel block its own output buffer and consolidating after, the per-block compute is short enough that we would saturate the DMA engine first. Each parallel block adds a full ≈ 9 MB weight read plus the gather/scatter traffic; the engine that owns those reads is already at >80% busy in our best version. The bottleneck is fundamentally bandwidth out of HBM rather than instruction throughput inside the cores.

## 5 Lessons

**Profile-driven, three-iteration design loop.** The single highest-leverage methodology choice we made part way through the competition was forcing every new version to come with a `neuron-profile` trace that showed the previous version's bottleneck and showed how it changed in the new one. Several attractive-looking optimizations (full softmax fusion when softmax is already 1% busy, indirect-DMA scatter at small $T$, column tiling at our specific shape) failed this test and saved us from committing to them.

**The biggest wins were structural, not instruction-level.** bf16 weights, the per-expert outer loop, active-expert compaction, and load+add+store instead of atomic_rmw account for the bulk of our end-to-end improvement. Instruction-level fusions registered as single-percent changes that compounded modestly but were not load-bearing. Of the structural moves, the per-expert loop was the single largest single change.

**Multi-LNC has a minimum-kernel-size threshold.** For our smaller kernels (RMSNorm at small num_rows, router at small $T$), the fixed per-LNC setup - each LNC reloads weights into its own SBUF, kernel entry/exit tax, host coordination overhead - exceeded the parallelism win. Multi-LNC paid off cleanly only past a kernel-specific shape threshold. The threshold scales with per-token compute work: smaller for the heavier blockwise body, larger for the lighter router.

## 6 Future work

- **Overlap the next expert's weight load with the current expert's compute.** The per-expert outer loop saved most of the redundant DMA, but each expert still pays a one-time stall while its weights stream in before any of its blocks can begin. A ring of SBUF buffers that pre-loads the next expert while the current one computes would hide that stall entirely, and on a DMA-bound kernel that is the most direct remaining win available. We hit a compiler issue trying to implement this pattern naively; production code achieves the same effect with a more elaborate weight layout and accumulator-bank rotation that side-steps it. Adopting that layout is the natural next step.
- **TensorE column-tiling that actually parallelises.** Splitting each matmul into four narrower matmuls running side-by-side on disjoint columns of the systolic array would, in principle, double matmul throughput on the per-block kernel. Our naive implementation serialized on accumulator bank access; rotating each split's output into a distinct bank should restore the parallelism. This is the lever for taking the win past what reducing weight bytes can buy.
- **FP8 weights with `double_row` matmul.** A further 2× theoretical matmul throughput on Trn2. Our submission already stores weights in fp8 but does not yet enable the double-row mode; the change is in the kernel scheduler, not the math, and would compound with the previous item.
- **Fused router + experts kernel.** Today the router writes the per-token expert affinities to HBM and the blockwise body reads them back. Fusing the two kernels would remove a layer-frequent HBM round-trip and open additional cross-stage pipelining opportunities.
- **NKI attention path.** We accepted the bundled `attention_isa_kernel`. A custom rewrite with paged KV-cache handling and explicit double-buffered DMA could move the next several percent of end-to-end latency, but the engineering cost is high relative to the prefill MoE wins still available.

## 7 Conclusion

We presented three NKI kernels for Qwen3-30B-A3B prefill inference on AWS Trainium - RMSNorm, top-K router, and blockwise expert MLP - developed through a profile-driven design process. Cumulatively the kernels deliver an ≈12% end-to-end throughput improvement on the MLSys 2026 Trainium contest harness, qualifying our team for round 2. The optimizations that mattered were primarily structural - bf16 weights with fp32 PSUM accumulation, a per-expert outer loop over the dispatch, active-expert compaction, and elimination of an unnecessary atomic scatter - rather than instruction-level fusions. The optimizations that did not move the needle were still good learning opportunities: they identify a structural ceiling, set by HBM-to-SBUF DMA bandwidth, that informs future work.

## Acknowledgments

We thank Prof. Christina Delimitrou for the course context that motivated this project. We also thank the AWS Neuron team for running the competition and providing the NKI documentation, simulator, and profiling tools that made this work tractable on a tight timeline.

## Code

All of our round 1 competition code is found here. All of our round 2 competition code is found in here.

## References

[1] Qwen Team. Qwen3-30B-A3B Model Card and Architecture Reference. *https://huggingface.co/Qwen/Qwen3-30B-A3B*, 2025.

[2] Amazon Web Services. AWS Trainium and Trainium2: Specifications and Programming Model. *AWS Neuron Documentation*, 2024–2026.

[3] Amazon Web Services. Neuron Kernel Interface (NKI) Documentation. *https://awsdocs-neuron.readthedocs-hosted.com/en/latest/nki/*, 2024–2026.

[4] Amazon Web Services. Fused Mamba on AWS Trainium: A Three-Iteration NKI Tutorial. *AWS Neuron Documentation*, 2025.

[5] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations*, 2017.

[6] Amazon Web Services. Neuronx-Distributed-Inference: BlockwiseMatmulNKIFunc and Production MoE Kernels. *AWS Neuron Documentation*, 2025–2026.